IMEN574 / IDSC574 · PROGRAMMING FOR AX

7

Data Validation and Evaluation

Sangwoo Mo · POSTECH
Spring 2027

Where We Are

01 Programming with AI

Lectures 2–4

02 **Python and Classical Machine Learning**

Lectures 5–13

03 Deep Learning and Foundation Models

Lectures 14–22

Outline: Data Validation and Evaluation

01 **Define Valid Inputs**

Protect the prediction boundary

02 **Design the Split**

Match evaluation to deployment

03 **Measure the Decision**

Choose metrics with a baseline

01 · Define Valid Inputs

01 **Define Valid Inputs**

Protect the prediction boundary

02 **Design the Split**

Match evaluation to deployment

03 **Measure the Decision**

Choose metrics with a baseline

Why Validation Design

A high score can measure the wrong future

VALIDATION CLAIM	AVAILABLE DATA
Predict faults on unseen machines	Repeated rows from the same machines
Decision happens before inspection	Repair notes written after inspection

The intended future determines the evaluation design

The Prediction Unit

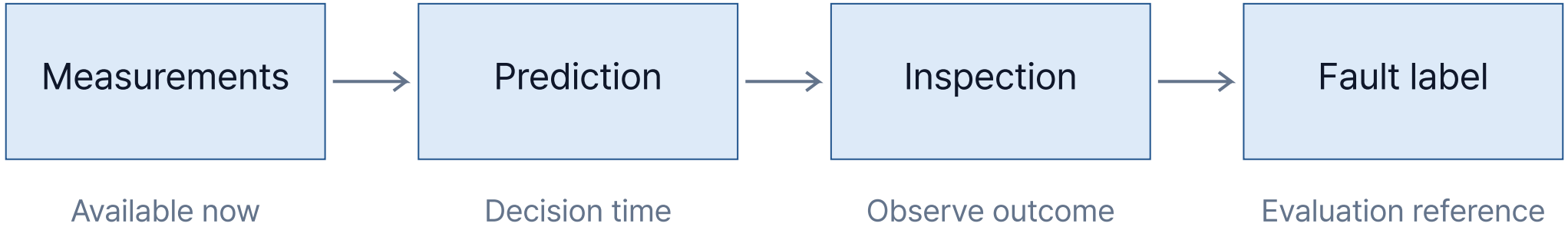
Define what one prediction refers to

Machine	Time	Decision Point
M1	08:00	before inspection
M1	09:00	before inspection
M2	08:00	before inspection

Repeated machine rows are not independent machines

Target Availability

The target must exist after the prediction is made



Features must be available at the decision time

Caveat: Target Leakage

An outcome field can reveal the answer

Machine	Score	Repair Action	Fault
M1	0.7	replaced	1
M2	0.2	none	0
M3	0.6	replaced	1

A post-outcome field can encode the target

https://scikit-learn.org/stable/common_pitfalls.html

Schema Checks

Make expected columns and ranges explicit

```
DATA ASSERTIONS

Required columns exist
One row means one measurement
Scores are finite and between 0 and 1
Keys are unique at the declared unit
The target is separate from the feature list
```

A schema check protects the declared task

Missingness as Evidence

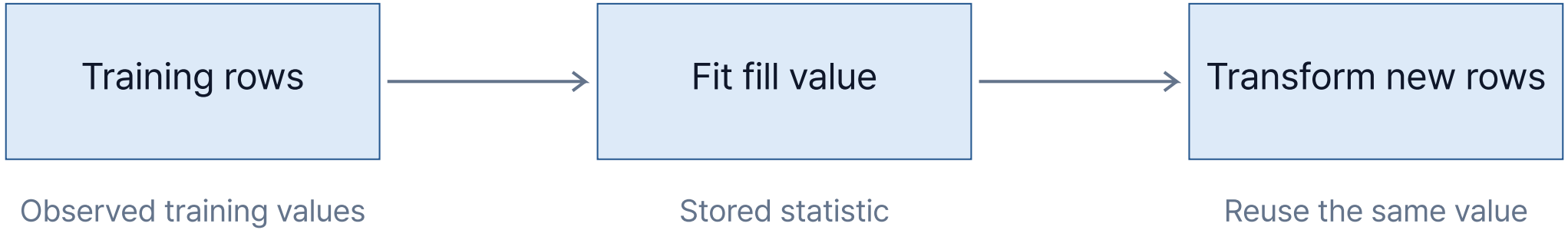
A missing pattern can differ across groups

Group	Rows	Missing Scores
Line A	4	0
Line B	4	2

Half of Line B is missing in this toy input

Imputation as a Learned Step

Estimate a fill value on training data



The fill value belongs to the fitted pipeline

Caveat: Preprocessing Leakage

A test value can move the learned mean



Adding the test value changes the learned statistic

https://scikit-learn.org/stable/common_pitfalls.html

A Frozen Transformation

Apply training statistics without refitting

```
import numpy as np
train = np.array([1., 2., 3.])
test = np.array([np.nan, 100.])
fill = train.mean()
filled = np.where(np.isnan(test), fill,
test)
print(fill)
print(filled)
```

```
2.0
[  2. 100.]
```

The test set receives the stored training fill value

Feature Availability Audit

Trace each column to its collection time

Feature	Known at Prediction?	Action
Sensor score	Yes	Keep
Machine ID	Yes	Match split to use
Repair note	No	Exclude
Inspection result	No	Target only

Feature inclusion depends on both timing and the task

A Valid Input Snapshot

Keep the data version and feature list

```
SNAPSHOT

Version: inspection-v1
Unit: machine-time measurement
Features: score, temperature
Target: future inspection fault
Excluded: repair notes and post-inspection fields
```

A score is interpretable only with its input definition

02 · Design the Split

01 Define Valid Inputs

Protect the prediction boundary

02 Design the Split

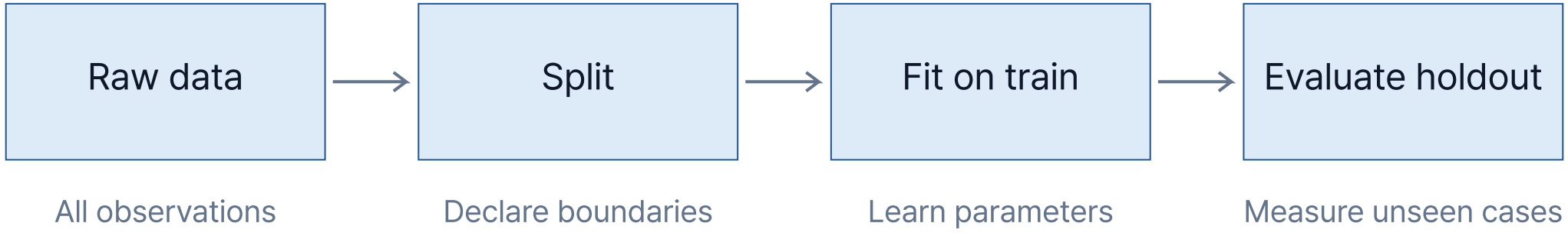
Match evaluation to deployment

03 Measure the Decision

Choose metrics with a baseline

Why Split Before Fitting

Test information must stay outside model decisions



The split creates an information boundary

https://scikit-learn.org/stable/common_pitfalls.html

Train, Validation and Test

Assign different jobs to different subsets

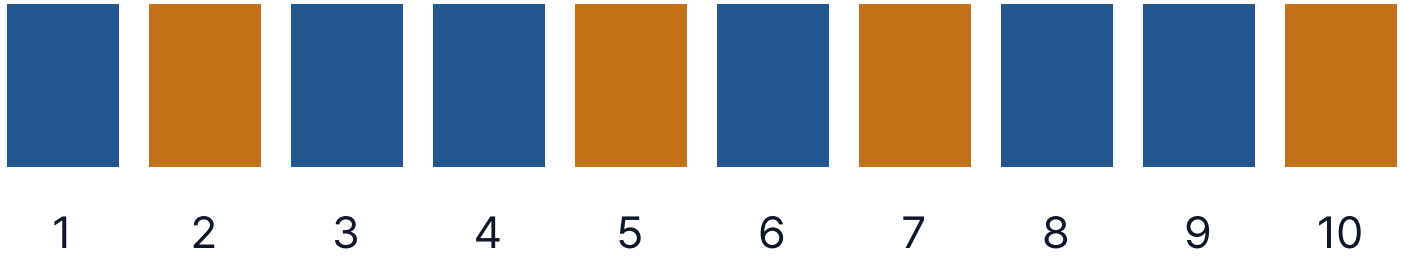
Subset	Job
Train	Learn parameters
Validation	Choose settings
Test	Estimate final performance

A subset role is defined by how it is used

Random Splits

Random rows assume the intended exchangeability

Rows randomly assigned



Blue: train rows

Orange: holdout rows

Random assignment alone does not remove dependence

Group Splits

Hold out the unit that must be new

Machine	Row 1	Row 2
M1	train	train
M2	train	train
M3	holdout	holdout

Every row from one machine stays on one side

Temporal Splits

Keep the test future later than the training past



Earlier observations

Choose settings

Later unseen outcomes

A time boundary tests a later deployment period

A Memorization Shortcut

A repeated ID can make a weak model look strong

ROW SPLIT	GROUP SPLIT
M1 appears in train and test	M1 appears only in test
Store M1 → fault label	No stored label for M1
Return the memorized label	Measure transfer to a new unit

The shortcut disappears when the identity is unseen

Cross-Validation

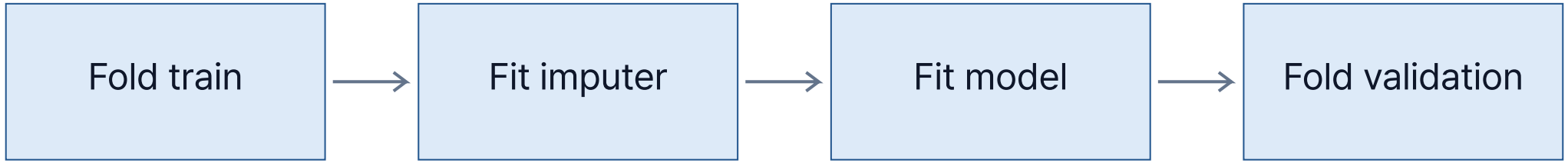
Repeat the train-validation roles inside development data

Run	Block A	Block B	Block C
Fold 1	valid	train	train
Fold 2	train	valid	train
Fold 3	train	train	valid

Each development block takes one turn as validation

A Fitted Pipeline

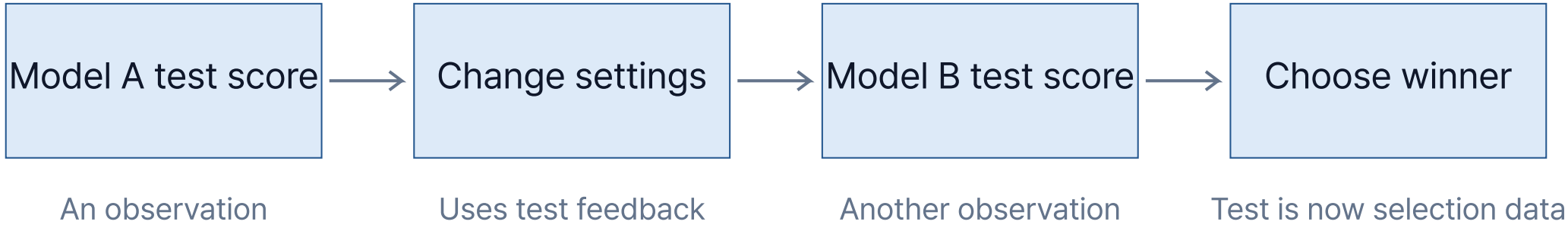
Refit transformations within each training fold



A transformation is fitted inside the fold boundary

Caveat: Repeated Test Peeking

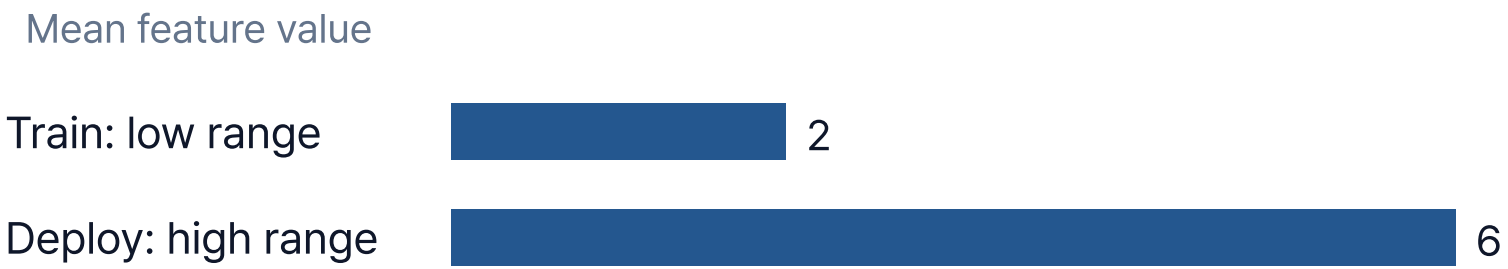
Repeated selection turns test feedback into training information



A repeatedly consulted test set loses its final-check role

Distribution Shift

A new population can change the relevant error



This toy deployment uses a shifted feature range

A Split Decision Record

State which future the score represents

```
EVALUATION DESIGN

Question: new machines in a later month
Hold out machine IDs and a later period
Fit preprocessing only on each training fold
Choose settings on validation data
Open the final test once after selection
```

The evaluation statement defines the score scope

03 · Measure the Decision

01 Define Valid Inputs

Protect the prediction boundary

02 Design the Split

Match evaluation to deployment

03 **Measure the Decision**

Choose metrics with a baseline

Why a Baseline

A model should beat a simple relevant alternative

Toy Dataset	Count
Negative	19
Positive	1
Always negative	19 correct

```
accuracy = 19 / 20 = 0.95
```

A high-accuracy baseline can miss every positive

Error Counts

Count before compressing the result into one number

	Predicted positive	Predicted negative
True positive	3	1
True negative	1	3

The matrix preserves two different error types

Precision and Recall Formulas

Each metric uses a different denominator

```
precision = TP / (TP + FP)
  recall = TP / (TP + FN)
```

Precision and Recall

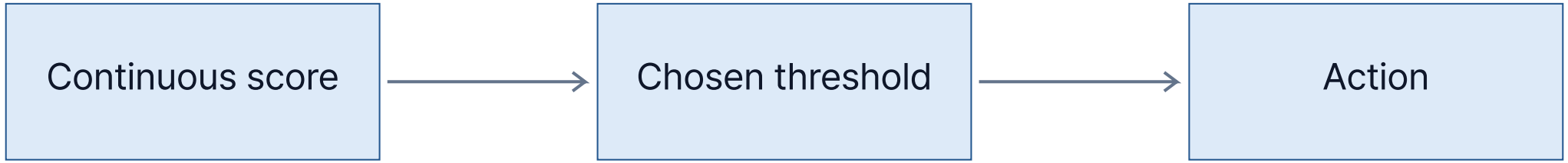
Each metric uses a different denominator



The two denominators answer different questions

Threshold Selection

The threshold converts scores into actions



A model output

Selected on validation

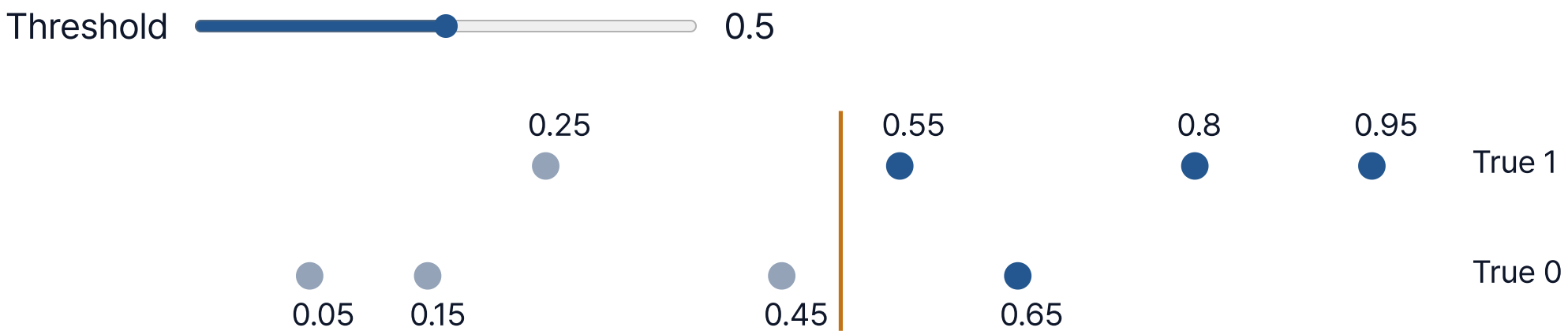
Inspect or release

Threshold choice belongs to the development procedure

Demo: Validation Threshold

Compare error types before selecting an operating point

Move the threshold; record TP, FP, FN and TN



Threshold 0.50 | TP 3, FP 1, FN 1, TN 3

Precision 0.75, recall 0.75, accuracy 0.75

Answer: Cost at the Default

Make the error cost explicit

$$\text{cost} = 1 \times \text{FP} + 5 \times \text{FN} = 1 + 5 = 6$$



One missed defect costs more under the stated assumption

Regression Metrics

Preserve units when explaining an error

Quantity	Value
Residuals	-1, 0, 1, 4
MAE	1.5
MSE	4.5
RMSE	2.121

The large residual contributes more strongly to squared error

Slice Evaluation

Averages can hide a weak subgroup

Slice	Correct	Rows	Accuracy
Line A	9	10	0.9
Line B	1	2	0.5
Overall	10	12	0.833

The overall score hides a small weak slice

Uncertainty and Sample Size

A small test gives a noisy estimate



The same fraction can carry different uncertainty

A Reproducible Scorecard

Publish the procedure with the score

```
SCORECARD

Target and prediction time
Data version, split rule and seed
Fitted preprocessing and selected settings
Baseline plus task-relevant metrics
Slice counts and remaining limitations
```

The procedure determines what the number means

Takeaway: Data Validation and Evaluation

01 Define what is available at prediction time

02 Fit only inside the training boundary

03 Read scores with baselines, slices and uncertainty